AI ASSISTED CODING

ASSIGNMENT-7.1

NAME : Shashikanth G

HT NO.: 2403A51245

BATCH NO: 11

TASK – 1

Prompt:

Syntax Error – Missing Parentheses in Print Statement.

Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., print "Hello"). Use AI to detect and fix the syntax error.

```
# Bug: Missing parentheses in print statement
def greet():
print "Hello, AI Debugging Lab!"
greet()
```

Requirements:

• Run the given code to observe the error.

• Apply AI suggestions to correct the syntax.

• Use at least 3 assert test cases to confirm the corrected code

works.

Expected Output 1:

• Corrected code with proper syntax and AI explanation

```python
# Bug: Missing parentheses in print statement

# Original buggy code:
# def greet():
#     print "Hello, AI Debugging Lab!"
# greet()

# AI Explanation:
# In Python 3, print is a function and requires parentheses. The original code uses Python 2 print statement syntax.
# To fix the syntax error, add parentheses to the print statement.

Tabnine | Edit | Test | Explain | Document
def greet():
    print("Hello, AI Debugging Lab!")

# Test cases to confirm the corrected code works
import io
import sys

Tabnine | Edit | Test | Explain | Document
def test_greet():
    captured_output = io.StringIO()
    sys.stdout = captured_output
    greet()
    sys.stdout = sys.__stdout__
    assert captured_output.getvalue().strip() == "Hello, AI Debugging Lab!"

# Additional asserts to check function call does not raise errors
try:
```

```python
    try:
        greet()
        no_error = True
    except Exception:
        no_error = False
    assert no_error

    # Assert greet returns None
    assert greet() is None

    print("All tests passed.")
```

OUTPUT:

```
[Running] python -u "c:\AI CODING\task-1"
Hello, AI Debugging Lab!
Hello, AI Debugging Lab!
All tests passed.

[Done] exited with code=0 in 0.319 seconds
```

TASK– 2

Logic Error – Incorrect Condition in If Statement

Prompt:

Task: Supply a function where an if-condition mistakenly uses = insteadof ==. Let AI identify and fix the issue.

# Bug: Using assignment (=) instead of comparison (==)
def check_number(n):
if n = 10:
return "Ten"
else:
return "Not Ten"

Requirements:

- Ask AI to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

Expected Output 2:

- Corrected code using == with explanation and successful test execution.

```
# Bug: Using assignment (=) instead of comparison (==)

# Original buggy code:
# def check_number(n):
#     if n = 10:
#         return "Ten"
#     else:
#         return "Not Ten"

# AI Explanation:
# In Python, '=' is the assignment operator and cannot be used in an if-condition.
# To compare values, use '=='. Using '=' in an if-statement causes a syntax error.

Tabnine | Edit | Test | Explain | Document
def check_number(n):
    if n == 10:
        return "Ten"
    else:
        return "Not Ten"

# Test cases to confirm the corrected code works
assert check_number(10) == "Ten"
assert check_number(5) == "Not Ten"
assert check_number(-10) == "Not Ten"

print("All tests passed.")
```

OUTPUT:

```
[Running] python -u "c:\AI CODING\task-2"
All tests passed.

[Done] exited with code=0 in 0.308 seconds
```

TASK – 3

Prompt:

Runtime Error – File Not Found

Task: Provide code that attempts to open a non-existent file and crashes.Use AI to apply safe error handling.

# Bug: Program crashes if file is missing
def read_file(filename):
with open(filename, 'r') as f:
return f.read()
print(read_file("nonexistent.txt"))

Requirements:

• Implement a try-except block suggested by AI.
• Add a user-friendly error message.
• Test with at least 3 scenarios: file exists, file missing, invalid path.

Expected Output 3:

• Safe file handling with exception management.

```
# Bug: Program crashes if file is missing

# Original buggy code:
# def read_file(filename):
#     with open(filename, 'r') as f:
#         return f.read()
# print(read_file("nonexistent.txt"))

# AI Explanation:
# Attempting to open a file that does not exist raises a FileNotFoundError.
# To handle this safely, use a try-except block and provide a user-friendly error message.

def read_file(filename):
    try:
        with open(filename, 'r') as f:
            return f.read()
    except FileNotFoundError:
        return f"Error: The file '{filename}' was not found."
    except Exception as e:
        return f"Error: {e}"

# Test cases
# 1. File exists
with open("testFile.txt", "w") as f:
    f.write("Test content.")
assert read_file("testfile.txt") == "Test content."
```

```
# 2. File missing
assert "was not found" in read_file("nonexistent.txt")

# 3. Invalid path
assert "Error:" in read_file("?:/invalid_path.txt")

print("All tests passed.")
```

OUTPUT:

```
All tests passed.

[Done] exited with code=0 in 0.299 seconds
```

TASK – 4

Prompt:

Attribute Error – Calling a Non-Existent Method

Task: Give a class where a non-existent method is called (e.g.,obj.undefined_method()). Use AI to debug and fix.

# Bug: Calling an undefined method
class Car:
def start(self):
return "Car started"
my_car = Car()
print(my_car.drive()) # drive() is not defined

Requirements:

• Students must analyze whether to define the missing method or correct the method call.
• Use 3 assert tests to confirm the corrected class works.

Expected Output #4:

• Corrected class with clear AI explanation.

```python
# Bug: Calling an undefined method

# Original buggy code:
# class Car:
#     def start(self):
#         return "Car started"
# my_car = Car()
# print(my_car.drive()) # drive() is not defined

# AI Explanation:
# The Car class does not have a drive() method, so calling my_car.drive() raises an AttributeError.
# To fix this, either define the drive() method or call an existing method (start()).
# Here, we define the missing drive() method.

class Car:
    def start(self):
        return "Car started"
    def drive(self):
        return "Car is driving"

my_car = Car()
print(my_car.drive())  # Now drive() is defined

# Test cases to confirm the corrected class works
assert my_car.start() == "Car started"
assert my_car.drive() == "Car is driving"
car2 = Car()
assert car2.drive() == "Car is driving"
print("All tests passed.")
```

OUTPUT:

```
[Running] python -u "c:\AI CODING\task-4"
Car is driving
All tests passed.

[Done] exited with code=0 in 0.281 seconds
```

TASK – 5

Prompt:

Type Error – Mixing Strings and Integers in Addition

Task: Provide code that adds an integer and string ("5" + 2) causing a TypeError. Use AI to resolve the bug.

Bug: TypeError due to mixing string and integer

def add_five(value):

return value + 5

print(add_five("10"))

Requirements:

• Ask AI for two solutions: type casting and string concatenation.
• Validate with 3 assert test cases.

Expected Output #5:

• Corrected code that runs successfully for multiple inputs.

```
# Bug: TypeError due to mixing string and integer
# Original buggy code:
# def add_five(value):
#     return value + 5
# print(add_five("10"))
# AI Explanation:
# Adding a string and an integer directly causes a TypeError in Python.
# Two solutions:
# 1. Type casting: Convert the input to an integer before addition.
# 2. String concatenation: Convert the integer to a string before concatenation.
# Solution 1: Type casting
Tabnine | Edit | Test | Explain | Document
def add_five_cast(value):
    return int(value) + 5
# Solution 2: String concatenation
Tabnine | Edit | Test | Explain | Document
def add_five_concat(value):
    return str(value) + "5"

# Test cases for type casting solution
assert add_five_cast("10") == 15
assert add_five_cast(2) == 7
assert add_five_cast("0") == 5

# Test cases for string concatenation solution
assert add_five_concat("10") == "105"
assert add_five_concat(2) == "25"
assert add_five_concat("abc") == "abc5"
print("All tests passed.")
```

OUTPUT:

```
[Running] python -u "c:\AI CODING\task-5"
All tests passed.
```